

Arrays



CONTENT

01 Introduction

02 Creation

03 Access

04 Iterating Arrays

05 Modifications

06 Array Methods





01

Introduction

Introduction

An Array is a special data structure in JavaScript used to store multiple values in a single variable

Example: `let fruits = ["Apple", "Banana", "Mango"];`
 `let mix = [1, "Hi", true, null];`



Why Use Arrays?

- Store multiple values together
- Easy to iterate (loop through)
- Supports many helpful **built-in methods**

Array Characteristics

Feature	Description
Index-based	Starts from index 0
Dynamic size	Can grow/shrink as needed
Mixed data types	Can store numbers, strings, objects etc

Special Notes :

Arrays are objects in JavaScript (`typeof [] === "object"`)

Arrays can hold objects, functions, or even other arrays (nested arrays)





02

Creation

Creating an Array

Array literal :

This is the most common way to create an array, using square brackets [] and placing the array elements inside the brackets, separated by commas.

```
let fruits = ['apple', 'banana', 'orange'];
```

Array Constructor :

This is the most common way to create an array, using square brackets [] and placing the array elements inside the brackets, separated by commas.

```
let numbers = new Array(3);    // creates an empty array with a length of 3
```

Note: It is recommended to create arrays using array literals instead of other methods.





03

Access

Access

Accessing array elements is possible using indices starting from 0 and incrementing by 1 for each subsequent element.

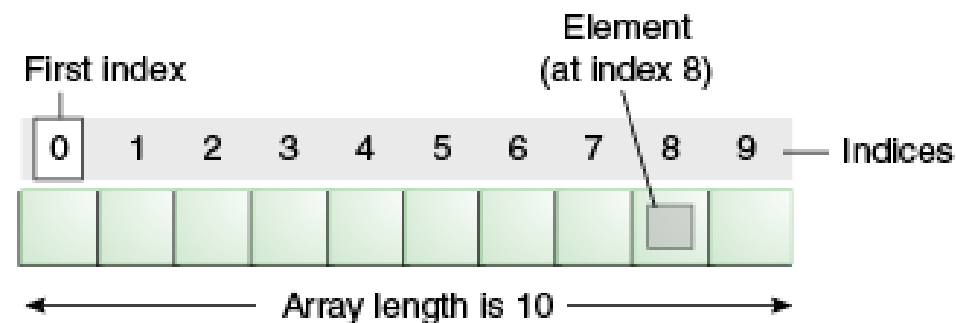
Example :

```
const fruits = ['apple', 'banana', 'orange'];
```

```
console.log(fruits[0]);    // output: "apple"
```

```
console.log(fruits[1]);    // output: "banana"
```

```
console.log(fruits[2]);    // output: "orange"
```





04

Iterating Arrays

Looping Through Arrays

Using for loop

```
let colors = ["Red", "Green", "Blue"];  
  
for (let i = 0; i < colors.length; i++) {  
  
  console.log(colors[i]);  
  
}
```

Output :

Red

Green

Blue



Looping Through Arrays

Using for...of :

The for...of loop is a modern, clean, and **easy way to iterate over iterable objects** — especially arrays, strings, sets, and maps.

Syntax :

```
for (let variable of iterable) {  
    // code block to execute  
}
```

Example :

```
let fruits = ["Apple", "Banana", "Mango"];  
for (let fruit of fruits) {  
    console.log(fruit);  
}
```

Output :

Apple

Banana

Mango





05

Modification

Modification in Arrays

JavaScript provides many ways to change (modify) arrays, such as:
Adding or removing items

- Replacing values
- Reordering elements
- Changing part of the array

Change a Value by Index

You can directly assign a new value at a specific index.

```
let colors = ["Red", "Green", "Blue"];
```

```
colors[1] = "Yellow";           // change "Green" to "Yellow"  
console.log(colors);           // ["Red", "Yellow", "Blue"]
```

Note :

JavaScript provides many **built-in array methods** that allow you to **modify arrays directly** — adding, removing, changing, or reordering elements.





06

Array Methods

Array Methods

Adding / Removing

Method	Description	Example
push()	Add to end	arr.push(4)
pop()	Remove from end	arr.pop()
unshift()	Add to beginning	arr.unshift(1)
shift()	Remove from beginning	arr.shift()

Searching :

Method	Description	Example
indexOf()	Finds index of element	arr.indexOf(20)
includes()	Checks if element exists	arr.includes("Red")



Array Methods

Transforming :

Method	Description	Example
join()	Combine into string	arr.join(", ")
toString()	Converts to string	arr.toString()
slice()	Returns portion (non-destructive)	arr.slice(1, 3)
splice()	Add/remove (destructive)	arr.splice(1, 2)
concat()	Merge arrays	arr1.concat(arr2)
reverse()	Reverses array	arr.reverse()
sort()	Sorts array	arr.sort()



Array Methods

Higher-Order Functions

Method	Description
map()	Creates new array from function
filter()	Returns matching items
reduce()	Reduces array to single value
find()	Returns first match
every()	Checks if all elements match condition
some()	Checks if any element matches condition

